

Here are the **basic Java OOP topics** you should learn first:

- **Class:** A blueprint for creating objects.
- **Object:** An instance of a class.
- **Encapsulation:** Wrapping data and methods together, usually with private fields and public getters/setters.
- **Abstraction:** Hiding implementation details and showing only essential features.
- **Inheritance:** One class acquires properties and methods from another class.
- **Polymorphism:** One action can behave differently in different situations.
- **Constructor:** A special method used to initialize objects.
- **Access modifiers:** public, private, protected, and default access control visibility.
- **this keyword:** Refers to the current object.
- **super keyword:** Refers to the parent class object.
- **Method overloading:** Same method name with different parameters.
- **Method overriding:** Child class provides a specific implementation of a parent method.
- **Interface:** A contract that a class implements.
- **Abstract class:** A class that cannot be instantiated and may contain abstract methods.
- **Association, aggregation, composition:** Different kinds of object relationships.

The core **four pillars** of OOP are abstraction, encapsulation, inheritance, and polymorphism. Java OOP lessons commonly also include classes, objects, constructors, access modifiers, interfaces, and abstract classes.

Easy study order

1. Class and object.
2. Constructor and access modifiers.
3. Encapsulation.
4. Inheritance.
5. Polymorphism.
6. Abstraction.
7. Interface and abstract class.
8. this, super, and object relationships.

Interview-friendly one-liners

- **Class:** template.
- **Object:** real instance.
- **Encapsulation:** data hiding.
- **Abstraction:** focus on what, not how.

- **Inheritance:** reuse parent code.
- **Polymorphism:** one name, many forms.

If you want, I can turn this into a **2-day Java OOP study plan** or give you **simple code examples for each topic**.

intermediate level topics

Completed 2 steps

For **intermediate Java OOP**, focus on concepts that go beyond the four pillars and basic class/object syntax. The most useful topics are inheritance types, method overriding, constructor chaining, interfaces, abstract classes, and object relationships like association, aggregation, and composition.

Topics to study

- **Method overloading vs overriding.**
- **Constructor overloading and constructor chaining.**
- **super keyword in inheritance.**
- **this keyword in constructors and methods.**
- **Interfaces and multiple inheritance through interfaces.**
- **Abstract classes vs interfaces.**
- **Runtime polymorphism.**
- **Upcasting and downcasting.**
- **Final keyword with variables, methods, and classes.**
- **Object relationships:** association, aggregation, composition.
- **Packages and access control in real projects.**
- **Nested classes and static nested classes.**

What matters most

The best intermediate topics for interviews are method overriding, interfaces, abstract classes, runtime polymorphism, and composition, because they are often used to test real understanding of OOP design in Java. Java also commonly emphasizes association, aggregation, and composition as extended OOP concepts beyond the core four pillars.

Study order

1. Overloading and overriding.
2. this and super.
3. Constructor chaining.
4. Interfaces.
5. Abstract classes.
6. Runtime polymorphism.
7. Upcasting and downcasting.

8. Final keyword.
9. Association, aggregation, and composition.
10. Packages and access modifiers.

Interview-style focus

- **Overloading:** same name, different parameters.
- **Overriding:** child class changes parent behavior.
- **Abstract class:** partial blueprint.
- **Interface:** contract for classes.
- **Composition:** strong has-a relationship.

I can also give you a **full Java OOP roadmap from basic to advanced** or a **topic-wise interview question list**.

java advance level topics

Here are the main **advanced Java topics** you should study after OOP basics:

Core advanced topics

- **Collections Framework:** List, Set, Map, Queue, iterators, sorting, and comparators.
- **Exception handling:** custom exceptions, checked vs unchecked exceptions, try-with-resources, chained exceptions.
- **Multithreading and concurrency:** threads, synchronization, locks, executors, thread safety.
- **File handling and I/O:** reading, writing, buffering, serialization, NIO.
- **Java 8+ features:** lambdas, streams, method references, functional interfaces, optional.
- **Generics:** type parameters, wildcards, bounded types.
- **JDBC:** database connectivity, queries, result sets, transactions.
- **Garbage collection and memory management:** heap, stack, GC basics, memory leaks.
- **Reflection:** inspecting and using classes, methods, and fields at runtime.
- **Annotations:** built-in and custom annotations, runtime metadata.
- **Serialization:** converting objects to bytes and back.
- **Design patterns:** Singleton, Factory, Builder, Strategy, Observer, etc.
- **Networking:** sockets, HTTP basics, client-server communication.
- **Modules and packages:** project structure and modularization.

Best order to learn

1. Collections.
2. Exception handling.
3. Generics.
4. File handling and I/O.

5. Java 8 features.
6. Multithreading.
7. JDBC.
8. Reflection and annotations.
9. Serialization.
10. Design patterns.

Interview-important topics

If your goal is interviews, focus first on **collections, exceptions, multithreading, Java 8 streams, JDBC, and design patterns**. These are the topics most often used to test deeper Java knowledge and practical coding ability